Document number	P3379R0
Date	2024-08-23
Project	Programming Language C++, Library Evolution Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Constrain std::expected equality operators

Revision History

- Initial revision

Introduction

P2944R3 (Comparisons for `reference_wrapper`) added constraints to the relational operators for `pair`, `tuple`, `optional`, and `variant`, but missed `expected` out. We should fix that.

Discussion

We needed LWG issue 4072 to fix the constraints for `std::optional`. We should do the same here.

This has been implemented and tested in libstdc++.

Proposed wording

The edits are shown relative to N4988.

Update the value of the `__cpp_lib_constrained_equality` feature test macro in [version.syn].

Change *Mandates*: to *Constraints*: in [expected.object.eq]:

22.8.6.8 Equality operators [expected.object.eq]

```
template<class T2, class E2> requires (!is_void_v<T2>)
  friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

-1- ~~*Mandates*~~ *Constraints*: The expressions `*x == *y` and `x.error() == y.error()` are well-formed and their results are convertible to `bool`.

-2- *Returns*: If `x.has_value()` does not equal `y.has_value()`, `false`; otherwise if `x.has_value()` is `true`, `*x == *y`; otherwise `x.error() == y.error()`.

```
template<class T2> friend constexpr bool operator==(const expected& x, const T2& v);
```

-3- ~~*Mandates*~~ *Constraints*: `T2` is not a specialization of `expected`. The expression `*x == v` is well-formed and its result is convertible to `bool`.

[*Note 1*: `T` need not be *Cpp17EqualityComparable*. — *end note*]

-4- *Returns*: `x.has_value() && static_cast<bool>(*x == v)`.

```
template<class E2> friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

-5- ~~*Mandates*~~ *Constraints*: The expression `x.error() == e.error()` is well-formed and its result is convertible to `bool`.

-6- *Returns*: `!x.has_value() && static_cast<bool>(x.error() == e.error())`.

Change *Mandates*: to *Constraints*: in [expected.void.eq]:

22.8.7.8 Equality operators [expected.void.eq]

```
template<class T2, class E2> requires is_void_v<T2>
  friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

-1- ~~*Mandates*~~ *Constraints*: The expression `x.error() == y.error()` is well-formed and its result is convertible to `bool`.

-2- *Returns*: If `x.has_value()` does not equal `y.has_value()`, `false`; otherwise `x.has_value() || static_cast<bool>(x.error() == y.error())`.

```
template<class E2>
  friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

-3- ~~*Mandates*~~ *Constraints*: The expression `x.error() == e.error()` is well-formed and its result is convertible to `bool`.

-4- *Returns*: `!x.has_value() && static_cast<bool>(x.error() == e.error())`.

References

N4988, Working Draft - Programming Languages -- C++, Thomas Köppe, 2024.
P2944R3, Comparisons for `reference_wrapper`, Barry Revzin, 2024.